

NetConnect AI: Personalized Networking Assistant

Project Description:

NetConnect AI is a personalized networking assistant that helps job seekers, students, and professionals craft high-quality outreach messages, icebreakers, and follow-ups for real-world networking situations. The platform combines a Streamlit front end with a FastAPI backend, and pairs the Google Gemini API for personalized message generation with a Wikipedia-based fact-checking layer that verifies factual claims about the people, companies, or events referenced in the conversation.

The application includes a Message Generator module, which produces a personalized outreach message, an icebreaker, and a follow-up draft for a given goal (cold outreach, informational interview request, post-event follow-up, or thank-you note), a Fact-Check module, which cross-references any factual statement in the generated message against Wikipedia and flags unverified or inaccurate claims, and a History module, which stores every interaction locally in a JSON file so the user can revisit, compare, and refine previous drafts.

Utilizing Google's Gemini API for natural-language generation, NetConnect AI processes the user's networking context (target person or company, purpose, tone, and key talking points) to produce ready-to-send content in seconds. The FastAPI backend exposes clean REST endpoints, while Streamlit renders a lightweight, form-based interface. With secure API key management via .env and structured JSON persistence for history, NetConnect AI empowers job seekers and professionals to network with confidence and accuracy.

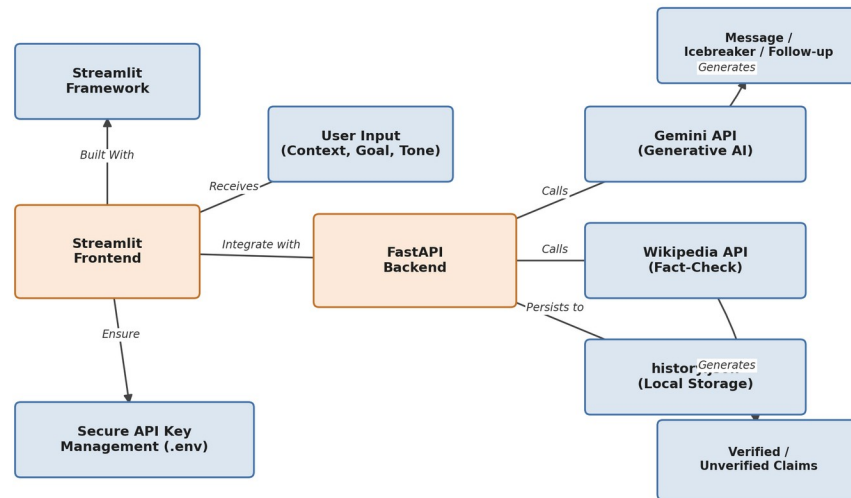
Scenarios

Scenario 1: A student preparing for a career fair enters the name of a target company and selects "Cold Outreach" with a Friendly tone. The system generates a personalized LinkedIn message, an icebreaker referencing the company's recent product launch, and a short follow-up draft, all fact-checked against Wikipedia in seconds.

Scenario 2: A job seeker wants to request an informational interview with a senior engineer at a target firm and selects "Informational Interview" with a Professional tone. NetConnect AI generates a respectful, value-driven request message, suggests two relevant talking points about the person's public work history, and verifies any company facts mentioned before presenting the draft.

Scenario 3: A professional who met a recruiter at a conference selects "Post-Event Follow-Up" with a Warm tone. The AI generates a message referencing the event, a personalized thank-you note, and a suggested next step (e.g., scheduling a call), while the Fact-Check module confirms the conference and organization details mentioned in the draft.

Technical Architecture:



Description: NetConnect AI utilizes a modular three-tier architecture to deliver accurate, AI-driven networking content. The Streamlit frontend provides a simple, form-based interface for entering networking context and tone. The FastAPI backend orchestrates input validation, prompt construction, and coordination between the Gemini API and the Wikipedia fact-checking service. Generated content and fact-check results are then persisted to a local JSON history file, and the final response is returned to the Streamlit interface for display.

(Note: Since this project uses a local JSON file rather than a relational database, no ER Diagram is required; the JSON schema used for history records is described in the Solution Architecture document.)

Pre-requisites:

- **Streamlit Framework Knowledge:** [Streamlit Documentation](#)
- **FastAPI Familiarity:** [FastAPI Documentation](#)
- **Google Gemini API Access:** [Gemini API Documentation](#)
- **Wikipedia API / wikipedia-api Library:** [Wikipedia API Documentation](#)
- **Python Programming Proficiency:** [Python Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Development Environment Setup:** [Streamlit Installation Guide](#)

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Generate a Google Gemini API key and configure it securely in the project environment.
- **Activity 1.2:** Research and select the appropriate Gemini model for personalized message generation.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the Streamlit frontend, FastAPI backend, Gemini API, and Wikipedia fact-checker.

- **Activity 1.4:** Set up the development environment, installing necessary libraries and dependencies for Streamlit, FastAPI, and the Gemini SDK.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core functionalities: Message Generator, Fact-Checker, and History Manager.
- **Activity 2.2:** Implement the FastAPI backend to manage routing and request processing, ensuring smooth coordination between Gemini and Wikipedia calls.

Milestone 3: Backend (main.py) Development

- **Activity 3.1:** Write the main API logic in main.py, establishing routes for each feature and integrating AI and fact-check responses.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the Streamlit user interface, ensuring a clean and intuitive layout.
- **Activity 4.2:** Render generated messages, fact-check results, and history entries dynamically based on user interactions.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring the server environment and ensuring all dependencies are correctly installed.
- **Activity 5.2:** Run the FastAPI backend and Streamlit frontend together locally, verifying end-to-end functionality.

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate Gemini model for our networking-message generation needs. This involves researching the capabilities and response quality of the available Gemini models, ensuring the chosen model aligns well with the application's objectives of generating outreach messages, icebreakers, and follow-ups across different goals and tones.

Activity 1.1: Generate a Gemini API Key

Before the application can communicate with the Gemini model, you need a valid API key. Follow the steps below to create one and connect it securely to the project.

- **Step 1 — Create a Google AI Studio Account:** Visit <https://aistudio.google.com> and sign in with your Google account.
- **Step 2 — Generate an API Key:** Navigate to the “Get API Key” section and create a new key for the project.
- **Step 3 — Add the Key to Your Project:** Create a .env file in the root of your project directory and paste the key as shown below:

```
GEMINI_API_KEY=your_copied_api_key_here
```

- **Step 4 — Verify the Key is Loaded:** The backend loads this key automatically at startup using python-dotenv:

```
from dotenv import load_dotenv
load_dotenv()
genai.configure(api_key=os.environ.get("GEMINI_API_KEY"))
```

- **Important — Never Commit Your API Key:** Add .env to your .gitignore file to prevent accidentally pushing the key to a public repository:

```
.env
```

Activity 1.2: Research and Select the Appropriate Generative AI Model

Here are the things that needed to be researched to select the best model for the project:

- **Understand the Project Requirements:** Review the specific needs of NetConnect AI, focusing on the types of content it will generate (outreach messages, icebreakers, and follow-ups).
- **Explore Gemini's Model Documentation:** Visit the Gemini documentation to examine the available models, including their context windows, speed, and limitations.
- **Evaluate Model Performance:** Compare models based on response quality, generation speed, and relevance to professional networking copy.
- **Select the Optimal Model:** Choose gemini-1.5-flash for its balance of response quality and low latency, set with a temperature of 0.7 for natural yet professional phrasing.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including the Streamlit frontend, FastAPI backend, Gemini API, and Wikipedia fact-checking service.
- **Detail Frontend Functionality:** Outline how users interact with the application entering networking context, selecting a goal (Cold Outreach, Informational Interview, Post-Event Follow-Up, or Thank-You), and choosing a tone (Professional, Friendly, or Warm).
- **Outline Backend Responsibilities:** Specify how the FastAPI backend handles incoming POST requests to /generate, validates and sanitizes user input, builds a structured prompt, and communicates with the Gemini API and Wikipedia service.
- **Describe AI Integration Points:** Define how the backend constructs prompts using GOAL_DESCRIPTIONS and TONE_DESCRIPTIONS dictionaries, calls the Gemini API, parses the JSON response via extract_json(), runs extracted factual claims through the fact_check() function, and returns structured results to the frontend.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.9+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\myenv\Scripts\activate"
(myenv) D:\projects>pip install -r requirements.txt
```

- **Install FastAPI and Uvicorn:** Use pip to install the backend dependencies:

```
(myenv) D:\projects> pip install fastapi==0.111.0 uvicorn==0.30.1
```

- **Install Streamlit:** Install the Streamlit library for the frontend:

```
(myenv) D:\projects> pip install streamlit==1.36.0
```

- **Install Gemini SDK and Wikipedia Library:** Install the Google Generative AI SDK and the Wikipedia API wrapper:

```
(myenv) D:\projects> pip install google-generativeai==0.7.2 wikipedia==1.4.0
```

- **Install python-dotenv:** Install dotenv support for secure API key management:

```
(myenv) D:\projects> pip install python-dotenv==1.0.1
```

- **Set Up Application Structure:** Create the project directory structure including folders for the Streamlit app, backend routes, and history storage (app.py, backend/main.py, requirements.txt, history.json).

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content Generation Features

Implement the three core generation features Message Draft, Icebreaker/Talking Points, and Fact-Check as a unified AI-assisted response driven by a single structured prompt and a verification pass.

- **Message Generator:** Generates a personalized outreach message, one icebreaker, and one follow-up draft. For Cold Outreach/Professional tone, produces concise, value-led copy. For Post-Event Follow-Up/Warm tone, produces a personable, appreciative message referencing the shared context.
- **Fact-Checker:** Extracts factual claims from the generated message (company names, roles, events) and cross-references each against Wikipedia, flagging any claim that cannot be verified.
- **History Manager:** Stores every generation request and its output (goal, tone, message, fact-check result, and timestamp) as a new entry in a local history.json file.

Activity 2.2: Implement the FastAPI Backend

Define Routes in FastAPI:

- Set up routes in backend/main.py for the health check and content generation endpoint.

Process User Input:

- Create a Streamlit form for users to submit their networking context, goal, and tone.
- Use FastAPI's Pydantic request models to validate JSON data submitted from the frontend.
- Validate that context is non-empty and within the 1500-character limit before processing.

Integrate API Calls:

- Within the /generate route handler, call the build_prompt() function to construct a structured prompt combining goal descriptions and tone descriptions.
- Submit the prompt to the Gemini API using model.generate_content() with the gemini-1.5-flash model.
- Parse and validate the JSON response using extract_json(), which handles direct JSON, markdown code blocks, and raw JSON object extraction as fallback strategies.
- Pass any extracted factual claims to fact_check(), which queries the Wikipedia library and returns a verified/unverified status for each claim.

Milestone 3: Backend (main.py) Development

Milestone 3 focuses on creating and refining the core logic of the backend, which serves as the backbone of the NetConnect AI application. In this milestone, you'll set up each feature's functionality through FastAPI routes, establish reliable prompt construction, Gemini interaction, and Wikipedia fact-checking, and conduct testing to ensure each feature performs as expected.

Activity 3.1: Writing the Main Application Logic in main.py

Define the Core Routes in main.py:

- Establish routes for NetConnect AI's two endpoints, each serving a distinct purpose:
 - /health — Health check route that confirms the backend and Gemini client are ready

```
@app.get("/health")
def health():
    return {"status": "ok"}
```

- /generate — POST endpoint that accepts JSON input, invokes the AI, fact-checks the result, and returns structured content

```
@app.post("/generate")
def generate(request: GenerateRequest):
    context = request.context.strip()
    goal = request.goal.lower()
    tone = request.tone.lower()

    if not context:
        raise HTTPException(status_code=400, detail="Networking context is required.")
    if len(context) > 1500:
        raise HTTPException(status_code=400, detail="Input too long. Please keep it under 1500 characters.")

    try:
        prompt = build_prompt(context, goal, tone)
        response = model.generate_content(prompt)
        result = extract_json(response.text)

        if not all(k in result for k in ["message", "icebreaker", "follow_up", "claims"]):
            raise ValueError("Invalid response structure from AI")

        fact_check_results = fact_check(result["claims"])
        save_history(context, goal, tone, result, fact_check_results)

        return {
            "success": True,
            "message": result["message"],
            "icebreaker": result["icebreaker"],
            "follow_up": result["follow_up"],
            "fact_check": fact_check_results,
            "goal": goal,
            "tone": tone,
        }
    except ValueError as e:
        raise HTTPException(status_code=500, detail=f"Failed to parse AI response: {str(e)}")
    except Exception as e:
```

```
raise HTTPException(status_code=500, detail=f"Generation failed: {str(e)}")
```

Set Up Route Handlers for Each Feature:

- For the /generate route, implement logic that reads context, goal, and tone from the incoming JSON body using a Pydantic request model.
- Apply input validation: return a 400 error if context is empty or exceeds 1500 characters.
- Route requests from the Streamlit frontend to the /generate endpoint, which processes data and returns a JSON response.

Define Prompt Engineering Helpers:

- **GOAL_DESCRIPTIONS** dictionary: maps cold_outreach, informational_interview, post_event_follow_up, and thank_you to detailed messaging instructions embedded in the prompt.

```
GOAL_DESCRIPTIONS = {
    "cold_outreach": "a concise, respectful first message to someone the user has not met, focused on a clear, low-pressure ask.",
    "informational_interview": "a polite request for a short conversation to learn about the person's role or career path, emphasizing genuine curiosity.",
    "post_event_follow_up": "a warm follow-up referencing a specific shared event or conversation, with a clear next step.",
    "thank_you": "a brief, sincere note of appreciation for someone's time or advice."
}
```

- **TONE_DESCRIPTIONS** dictionary: maps professional, friendly, and warm tones to style guidance for the model.

```
TONE_DESCRIPTIONS = {
    "professional": "formal, respectful, and business-appropriate.",
    "friendly": "approachable, upbeat, and conversational, while staying respectful.",
    "warm": "personable and appreciative, appropriate for someone the user has already met."
}
```

- **build_prompt()**: assembles a structured prompt string from the context, goal description, and tone description, instructing the model to return strictly valid JSON with a message, icebreaker, follow_up, and a list of factual claims used.

```
def build_prompt(context: str, goal: str, tone: str) -> str:
    goal_desc = GOAL_DESCRIPTIONS.get(goal, "a professional networking message")
    tone_desc = TONE_DESCRIPTIONS.get(tone, "professional")
    return f"""You are an expert networking coach and communication strategist.
Write {goal_desc} in a {tone_desc} tone, based on this context:
{context}
Return ONLY valid JSON with keys: message, icebreaker, follow_up, claims (a list of factual statements you used)."""
```

Integrate the Gemini and Wikipedia Responses in Each Function:

- Call model.generate_content() with the assembled prompt, requesting JSON-only output.
- **extract_json()**: a robust parser that first attempts direct json.loads(), then strips markdown code fences (```json blocks), and finally uses a regex to locate a raw JSON object, ensuring reliable parsing across model response variations.

```
def extract_json(text: str) -> dict:
    """Extract JSON from LLM response, handling markdown code blocks."""
    try:
        return json.loads(text)
    except json.JSONDecodeError:
        pass
    match = re.search(r'```(?:json)?\s*([\s\S]*?)```', text)
    if match:
        try:
            return json.loads(match.group(1).strip())
        except json.JSONDecodeError:
            pass
    match = re.search(r'\{[\s\S]*\}', text)
    if match:
        try:
            return json.loads(match.group(0))
        except json.JSONDecodeError:
            pass
    raise ValueError("Could not extract valid JSON from response")
```

- **fact_check():** takes the list of claims returned by the model and searches Wikipedia for each one, marking it “Verified” if a matching page/summary is found and “Unverified” otherwise.

```
def fact_check(claims: list) -> list:
    results = []
    for claim in claims:
        try:
            summary = wikipedia.summary(claim, sentences=1)
            results.append({"claim": claim, "status": "Verified", "reference":
summary})
        except Exception:
            results.append({"claim": claim, "status": "Unverified", "reference":
None})
    return results
```

- Validate that the parsed result contains all required keys — message, icebreaker, follow_up, and claims — before returning it to the frontend.
- **Backend routes:** /health → confirms server status; /generate (POST) → accepts JSON, returns {success, message, icebreaker, follow_up, fact_check, goal, tone}

Milestone 4: Frontend Development

Milestone 4 focuses on developing a clean, user-friendly interface for NetConnect AI using Streamlit. This involves designing a simple, form-based layout and rendering the generated message, icebreaker, follow-up, and fact-check results dynamically based on user input.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base Streamlit Layout:

- Develop the main app.py file as the single-page interface, including a header with the NetConnect AI name and tagline, an input section, and a results section.
- Use Streamlit layout containers (st.columns, st.expander) to keep the structure organized and easy to scan.
- Use Streamlit's built-in components (text area, select boxes, buttons) for a clean, consistent look without custom CSS.

Design a Simple, Readable Layout:

- Organize the results into a two-column layout: the generated message and icebreaker on the left, the follow-up draft and fact-check results on the right.
- Use `st.success`, `st.warning`, and `st.info` blocks to visually distinguish verified claims, unverified claims, and general tips.

Create Separate UI Components for Each Output Type:

- **Message Card:** displays the generated outreach message with a copy-to-clipboard button (via `st.code`) for easy pasting into LinkedIn or email.
- **Icebreaker & Follow-Up:** displayed as clearly labeled text blocks beneath the main message.
- **Fact-Check Panel:** lists each extracted claim with a Verified/Unverified badge and a short reference summary where available.
- **History Sidebar:** displays the five most recent generations pulled from `history.json`, so the user can revisit past drafts.

Activity 4.2: Creating Dynamic Content Rendering

Handle Form Submission:

- Attach the Generate button to a handler that sends the form data as a JSON POST request to the FastAPI `/generate` endpoint using the `requests` library.
- During processing, show a Streamlit spinner (`st.spinner`) in place of the button while waiting for the response.

Render Results Dynamically:

- Implement a `render_results(data)` function that unpacks the API response and populates the message, icebreaker, follow-up, and fact-check sections.
- After rendering, automatically append the new entry to the History sidebar so the most recent draft appears first.

Restore UI State After Generation:

- Re-enable the Generate button and clear the spinner regardless of success or failure, and surface any backend error message via `st.error`.

Milestone 5: Deployment

In Milestone 5, the focus is on deploying NetConnect AI first locally to verify correct operation, and then documenting how it can be exposed publicly if needed for the internship demo. This involves configuring the server environment, managing dependencies, and launching both the backend and frontend for real-world use.

Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Create and activate a virtual environment, then install all required dependencies from `requirements.txt`:

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\myenv\Scripts\activate"
(myenv) D:\projects>pip install -r requirements.txt
```

Configure Environment Variables:

- Create a .env file in the project root and add your Gemini API key. The backend loads this automatically using python-dotenv at startup:

```
GEMINI_API_KEY=your_gemini_api_key_here
```

Activity 5.2: Local Testing and Verification

Start the FastAPI Backend:

- Run the backend locally using:

```
(myenv) D:\projects>uvicorn backend.main:app --reload --port 8000
```

Start the Streamlit Frontend:

- In a second terminal, run the Streamlit app:

```
(myenv) D:\projects>streamlit run app.py
```

- Once running, open a browser and navigate to “http://localhost:8501” to access the app, with the API available at “http://localhost:8000”.
- Interact with the assistant test all four goals (Cold Outreach, Informational Interview, Post-Event Follow-Up, Thank-You) and all three tones (Professional, Friendly, Warm) to verify the AI responses and fact-check results are correctly generated, parsed, and rendered.

Important Notes:

- **Two processes required:** the FastAPI backend and the Streamlit frontend must both be running simultaneously for the app to function.
- **History persistence:** history.json is created automatically on first run and grows with each generation; back it up if you want to preserve a demo history.
- **Wikipedia rate limits:** the wikipedia library makes live calls per claim, so very long inputs with many claims may take a few extra seconds to fact-check.

Exploring the Website's Web Pages:

Home / Generator Page:

Description: The single-page Streamlit interface of NetConnect AI serves as both the landing page and the generation tool. It features a clean header displaying the NetConnect AI name and tagline, followed by the input form, goal selector, tone selector, and the AI-generated results panel, all on one seamless page.

Input Section:

Description: A simple form containing the main generation inputs. Users enter their networking context (target person, company, or event) in a text area (up to 1500 characters), select their goal via a dropdown (Cold Outreach, Informational Interview, Post-Event Follow-Up, or Thank-You), choose their tone (Professional, Friendly, or Warm), and click the Generate button. While the AI processes the request, a spinner is shown in place of the button.

Results Section:

Description: Revealed dynamically after generation, this section displays the AI output in a two-column layout. The left column shows the generated message and icebreaker in labeled text blocks with a copy option. The

right column contains the follow-up draft and the fact-check panel, listing each factual claim with a Verified or Unverified badge. A History sidebar on the left of the page lists the five most recent generations pulled from history.json.

Conclusion:

NetConnect AI is a personalized networking assistant built with Streamlit and FastAPI that streamlines professional outreach while safeguarding factual accuracy. It uses Google's Gemini API to instantly generate tailored outreach messages, icebreakers, and follow-ups across cold outreach, informational interview, post-event follow-up, and thank-you scenarios, and pairs every generation with a Wikipedia-based fact-check pass and a locally persisted history log. The project, which included model selection, backend and frontend development, and local deployment, highlights how targeted AI combined with lightweight fact verification can boost confidence and accuracy for job seekers and professionals navigating real-world networking, with clear potential for future scalability. Looking ahead, NetConnect AI offers significant opportunities for expansion, such as integrating LinkedIn profile parsing for richer context, adding multi-language support, and implementing user accounts to sync history across devices. By continuously refining the underlying prompts and expanding the fact-checking sources beyond Wikipedia, the platform is well-positioned to evolve from a personal tool into a comprehensive networking assistant for job seekers and career centers alike.